

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN CÁ NHÂN
PHÁT TRIỂN ỨNG DỤNG CHO
THIẾT BỊ DI ĐỘNG (MAD)
ỨNG DỤNG PODY

*Nền tảng nghe podcast và tạo nội dung
podcast hỗ trợ AI*

Giảng viên hướng dẫn: *ThS. Nguyễn Hoàng Anh*

Nhóm QLĐT: *CNPM05*

Nhóm BTL: *01*

Sinh viên thực hiện: *Vũ Đức Thành – B22DCCN801*

Nhóm thành viên BTL chính:

STT	Họ và tên	MSSV	Phân công chính
1	Lại Xuân Hiếu	B22DCCN309	AI podcast (ai-service)
2	Vũ Đức Thành	B22DCCN801	Báo & Embedding (article, embed- ding)
3	Nguyễn Việt Huy	B22DCCN393	Hạ tầng & Auth (gateway, identity)
4	Lê Minh Ngọc	B22DCCN586	Thông báo (notification-service)

Hà Nội, 2026

MỤC LỤC

1	Tổng quan phạm vi phụ trách	4
1.1	Mục tiêu	4
1.2	Danh sách chức năng được phân công	4
1.3	Đầu vào, đầu ra và xử lý lỗi	5
2	Kiến trúc kỹ thuật	6
2.1	Thành phần chính	6
2.2	Luồng xử lý chính	7
2.3	Mô hình dữ liệu	9
3	Code đáp ứng chức năng	10
3.1	Module và file chính	10
3.2	API handler và hàm chính	11
3.3	Bảng cơ sở dữ liệu	11
3.4	API nội bộ và API gọi ngoài	13
4	Triển khai và kiểm thử	14
4.1	Yêu cầu môi trường	14
4.2	Các bước triển khai	14
4.3	Các bước kiểm thử	15
4.4	Lưu ý triển khai	17
5	Thông tin source code	19
5.1	Thông tin source code	19
5.2	Các file cá nhân liên quan	19
	Kết luận	20

DANH SÁCH HÌNH VẼ

2.1	Sơ đồ component cho article_service và embedding_service . .	6
2.2	Sequence đọc bài, sinh embedding và đồng bộ category ngữ nghĩa	7
2.3	Sequence tìm kiếm bài báo theo ngữ nghĩa	8
2.4	ERD rút gọn cho dữ liệu article và embedding	9

DANH SÁCH BẢNG

1.1	Chức năng thuộc phạm vi article và embedding	4
2.1	Thành phần trong phạm vi báo và embedding	6
3.1	Các file/module chính	10
3.2	Handler/hàm quan trọng	11
3.3	Bảng dữ liệu chính	11
3.4	API trong phạm vi phụ trách	13
5.1	Thông tin repository	19
5.2	File/folder liên quan trực tiếp	19

CHƯƠNG 1

TỔNG QUAN PHẠM VI PHỤ TRÁCH

1.1 Mục tiêu

Báo cáo này trình bày phần chức năng báo trong hệ thống Pody, tập trung vào hai service chính: `article_service` và `embedding_service`. Phạm vi bao gồm crawl và hiển thị bài báo, quản lý category, tương tác người dùng với bài viết, tạo podcast từ bài báo, sinh embedding, tìm kiếm ngữ nghĩa và đồng bộ category ngữ nghĩa về service bài báo.

1.2 Danh sách chức năng được phân công

Bảng 1.1: Chức năng thuộc phạm vi article và embedding

STT	Chức năng	Mô tả ngắn	Service/Module	Trạng thái
1	Crawl bài báo RSS	Lấy bài viết từ nguồn RSS, chuẩn hóa metadata, chống trùng URL và lưu vào PostgreSQL.	<code>article_service</code>	Hoàn thành
2	Feed và chi tiết bài báo	Cung cấp API danh sách, lọc theo category/từ khóa, xem chi tiết và tăng lượt xem.	<code>article_service</code>	Hoàn thành
3	Category và sở thích đọc	Trả danh sách category, lưu category yêu thích của người dùng, ưu tiên feed cá nhân.	<code>article_service</code>	Hoàn thành
4	Reaction, comment, metric	Ghi nhận LIKE/LOVE/DISLIKE, bình luận và thời gian đọc bài.	<code>article_service</code>	Hoàn thành
5	Podcast từ bài báo	Tạo job podcast từ nhóm bài đã chọn hoặc gợi ý, sinh kịch bản và audio.	<code>article_service/ai_podcast</code>	Hoàn thành
6	Embedding bài báo	Consume event bài báo, chia chunk, gọi Gemini embedding và lưu vector vào pgvector.	<code>embedding_service</code>	Hoàn thành

7	Tim kiếm ngữ nghĩa	Nhận query, embed query và tìm chunk bài báo gần nghĩa nhất bằng cosine similarity.	embedding_service	Hoàn thành
8	Gán category ngữ nghĩa	So khớp vector bài báo với vector category, publish event và cập nhật bảng category_articles.	Hai service	Hoàn thành

1.3 Đầu vào, đầu ra và xử lý lỗi

Đầu vào chính của `article_service` là request HTTP từ ứng dụng Flutter/API Gateway, dữ liệu RSS và thông tin xác thực từ header `X-Auth-User-ID`. Đầu ra là JSON feed bài báo, chi tiết bài, category, reaction, comment, metric và trạng thái job podcast. Service trả lỗi 400 khi dữ liệu không hợp lệ, 404 khi bài báo hoặc job không tồn tại, 500 khi lỗi hệ thống.

Đầu vào chính của `embedding_service` là event Kafka `article.embedding.requested`, request tìm kiếm semantic và cấu hình Gemini API key. Đầu ra là vector trong PostgreSQL/pgvector, kết quả search và event `article.category.matches.generated`. Service bỏ qua bài không public, tránh xử lý lại khi `content_hash` không đổi, ghi trạng thái job khi lỗi và trả 503 nếu provider/vector store chưa sẵn sàng.

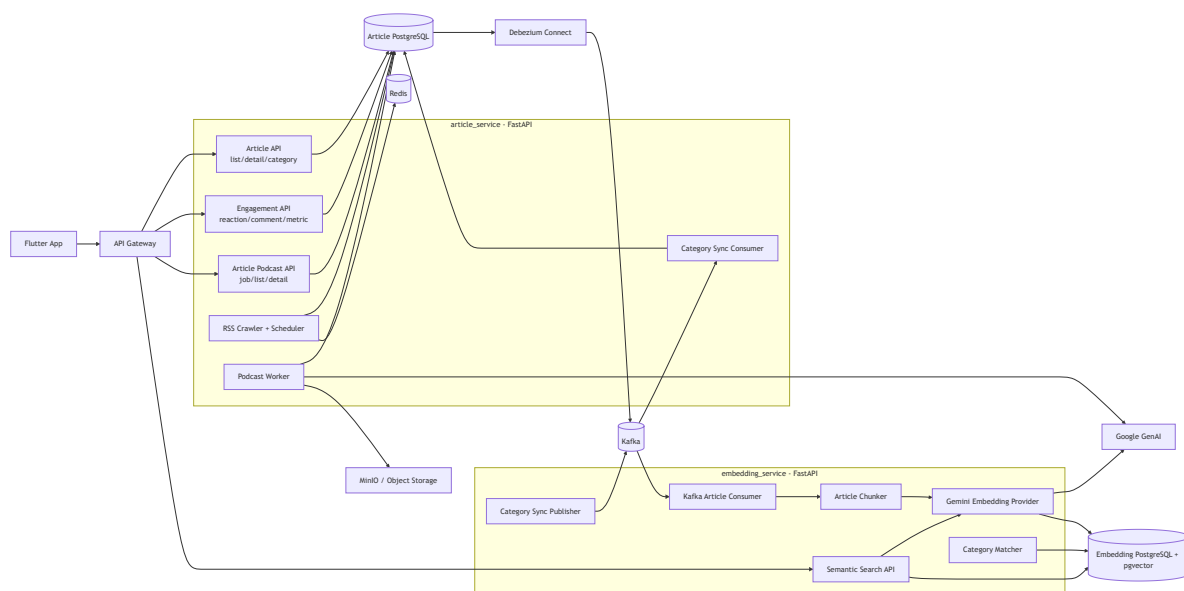
CHƯƠNG 2

KIẾN TRÚC KỸ THUẬT

2.1 Thành phần chính

Bảng 2.1: Thành phần trong phạm vi báo và embedding

Thành phần	Vai trò	Công nghệ	Kết nối
Flutter App	Giao diện đọc báo, chi tiết, podcast báo	Flutter	API Gateway
API Gateway	Điều phối request, truyền auth context	Go	article_service, embedding_service
article_service	Quản lý bài báo, category, tương tác và podcast báo	Python/FastAPI	Article DB, Redis, Kafka, Gemini, MinIO
embedding_service	Pipeline vector hóa và semantic search	Python/FastAPI	Embedding DB, Article DB, Kafka, Gemini
Debezium/Kafka	Chuyển đổi thay đổi dữ liệu bài báo thành event	Debezium/Kafka	Article DB, embedding consumer
PostgreSQL/pgvector	Lưu dữ liệu nghiệp vụ và vector embedding	PostgreSQL	Các repository của service



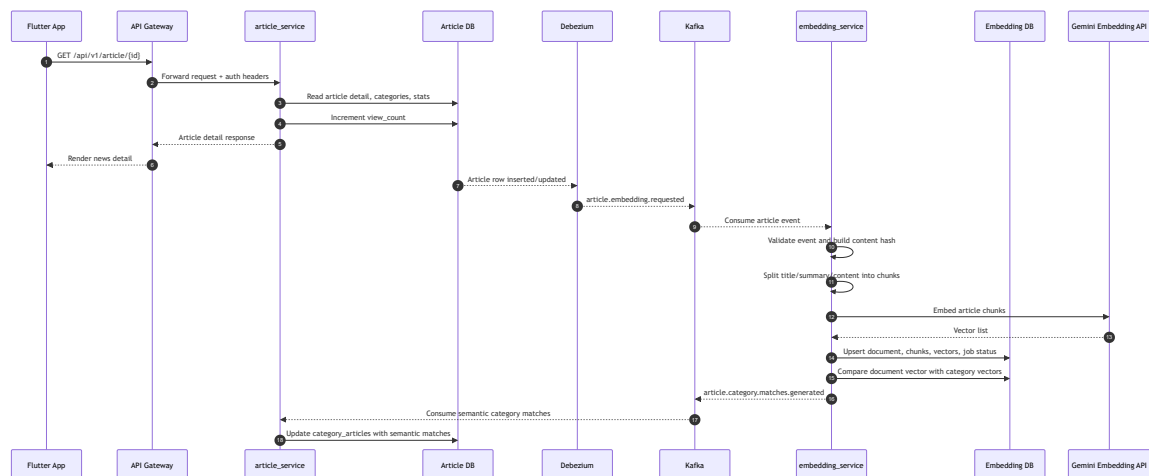
Hình 2.1: Sơ đồ component cho article_service và embedding_service

Sơ đồ component mô tả cách phân bổ của Pody được tách thành hai service chính. `article_service` chịu trách nhiệm với dữ liệu nghiệp vụ của bài

báo: crawl RSS, cung cấp API danh sách/chi tiết, ghi nhận reaction, comment, metric đọc bài, tạo podcast từ bài báo và nhận kết quả đồng bộ category ngữ nghĩa. `embedding_service` đứng ở phía xử lý ngữ nghĩa: nhận event bài báo từ Kafka, chia nội dung thành chunk, gọi Gemini để sinh embedding, lưu vector vào PostgreSQL có pgvector, phục vụ semantic search và publish kết quả gán category về Kafka.

Luồng kết nối quan trọng trong sơ đồ là đường từ Article DB sang Debezium, Kafka rồi đến embedding consumer. Nhờ vậy, khi bài báo được thêm hoặc cập nhật, hệ thống không cần gọi đồng bộ trực tiếp giữa hai service mà dùng event để xử lý bất đồng bộ. Cách này giúp `article_service` vẫn phản hồi nhanh cho client, còn `embedding_service` có thể xử lý tác vụ nặng như gọi AI API và tính toán vector ở nền.

2.2 Luồng xử lý chính

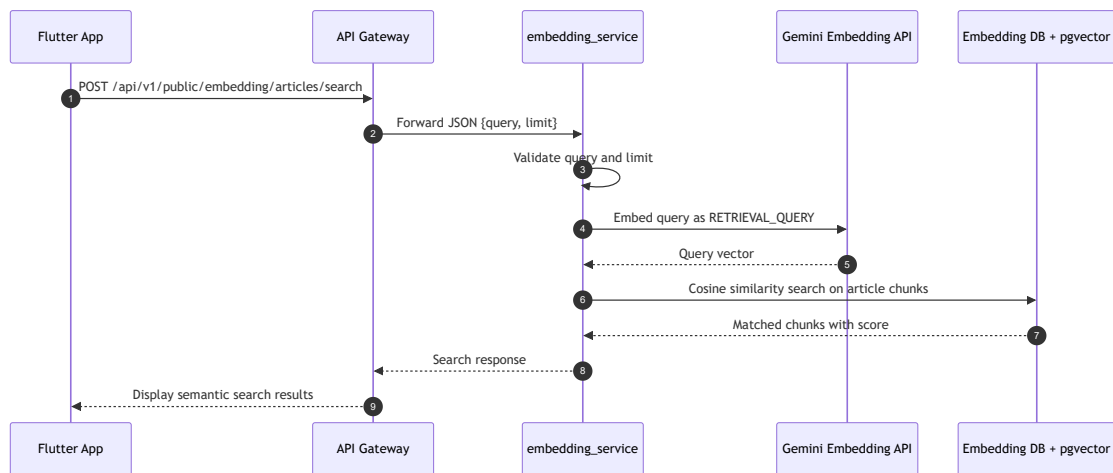


Hình 2.2: Sequence đọc bài, sinh embedding và đồng bộ category ngữ nghĩa

Sequence này thể hiện hai luồng xảy ra quanh một bài báo. Ở luồng người dùng, Flutter App gọi API Gateway để lấy chi tiết bài; gateway chuyển request đến `article_service`; service đọc dữ liệu từ Article DB, tăng `view_count`, sau đó trả response gồm nội dung bài, category, reaction và số bình luận.

Ở luồng nền, khi dữ liệu bài báo thay đổi trong Article DB, Debezium phát hiện thay đổi và đẩy event lên Kafka topic `article.embedding.requested`.

embedding_service consume event này, validate payload, tạo content_hash, chia bài thành các chunk, gọi Gemini Embedding API để sinh vector, rồi lưu document/chunk/vector/job vào Embedding DB. Sau đó service so khớp vector bài viết với vector category và publish event article.category.matches.generated; article_service consume event này để cập nhật bảng category_articles.

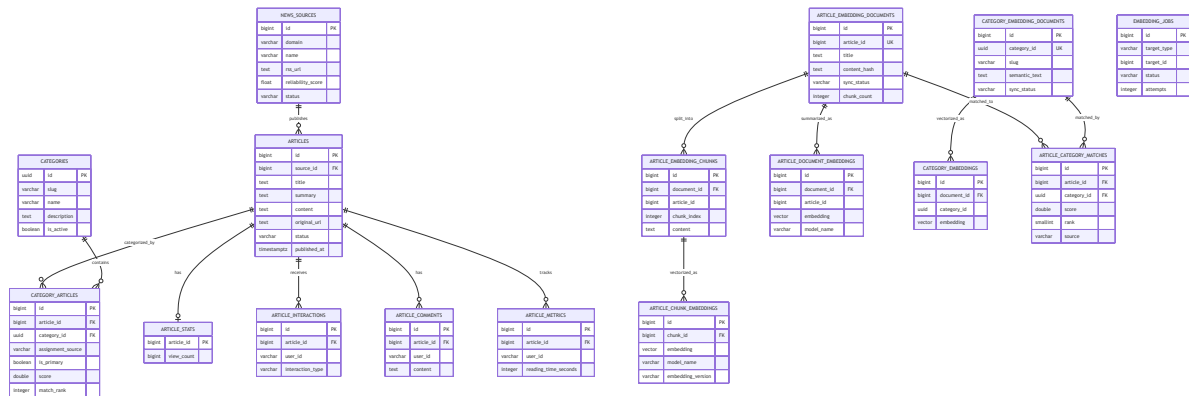


Hình 2.3: Sequence tìm kiếm bài báo theo ngữ nghĩa

Sequence tìm kiếm ngữ nghĩa mô tả luồng request POST /api/v1/public/embedding/articles/search. Người dùng nhập một câu truy vấn tự nhiên, ví dụ “tin tức công nghệ AI”. Gateway chuyển request đến embedding_service; service validate query và limit, gọi Gemini để sinh query vector với task type RETRIEVAL_QUERY, rồi dùng vector này truy vấn các chunk bài báo trong pgvector bằng cosine similarity.

Kết quả trả về là danh sách chunk gần nghĩa nhất, gồm article_id, tiêu đề, URL gốc, chỉ số chunk, đoạn preview và điểm tương đồng. Thiết kế này giúp tìm kiếm không chỉ phụ thuộc vào từ khóa chính xác trong tiêu đề/tóm tắt, mà có thể tìm các bài có nội dung tương đồng về mặt ngữ nghĩa.

2.3 Mô hình dữ liệu



Hình 2.4: ERD rút gọn cho dữ liệu article và embedding

ERD được chia thành hai cụm dữ liệu. Cụm Article DB là dữ liệu nghiệp vụ chính, gồm articles, news_sources, categories, category_articles, article_stats, article_interactions, article_comments và article_metrics. Trong đó articles là bảng trung tâm; category_articles là bảng nối giữa bài báo và category, đồng thời lưu thêm nguồn gán category là manual hay semantic.

Cụm Embedding DB lưu dữ liệu phục vụ AI/search, gồm document bài báo, chunk bài báo, vector từng chunk, vector tổng hợp document, document category, vector category, kết quả match category và bảng job/outbox. Các bảng vector dùng pgvector để hỗ trợ truy vấn similarity. Việc tách Article DB và Embedding DB giúp dữ liệu nghiệp vụ không bị phụ thuộc trực tiếp vào chi tiết triển khai AI, đồng thời cho phép tối ưu riêng các index vector.

Do hai service áp dụng mô hình database per service, sơ đồ không nối trực tiếp bảng articles với article_embedding_documents. Trường article_id trong Embedding DB chỉ được xem là mã tham chiếu nghiệp vụ nhận từ event Kafka/Debezium, không phải khóa ngoại vật lý sang Article DB.

CHƯƠNG 3

CODE ĐÁP ỨNG CHỨC NĂNG

3.1 Module và file chính

Bảng 3.1: Các file/module chính

File/Folder	Module	Chức năng	Giải thích
article_service/ api.py	FastAPI routes	API bài báo	Định nghĩa endpoint feed, detail, category, reaction, metric, comment.
article_service/ dependencies.py	Dependency wiring	DB/auth/service	Tạo session, đọc auth header, khởi tạo service nghiệp vụ.
services/ article_query_service.py	Service layer	Đọc dữ liệu	Ghép dữ liệu bài, category, stats, reaction và comment thành response.
services/ article_engagement _service.py	Service layer	Tương tác	Validate bài viết, reaction type, comment và metric.
services/ crawler/	Crawler pipeline	Crawl RSS	Discovery, extract metadata, storage và chống trùng.
ai_podcast/	Podcast báo	AI podcast	Tạo job, research, viết script, TTS, lưu audio.
services/ category_sync _consumer.py	Kafka consumer	Đồng bộ category	Nhận match semantic từ embedding service và cập nhật Article DB.
embedding_service/ app/runtime.py	Runtime	Điều phối pipeline	Khởi tạo repository, provider, consumer, publisher và health state.
service/ article_embedding _service.py	Service layer	Embedding bài báo	Xử lý event, chunk, gọi Gemini, lưu vector và search.
service/ category_embedding _service.py	Service layer	Embedding category	Bootstrap category từ Article DB và sinh vector category.
repository/ embedding_repository.py	Repository	pgvector	Upsert document/chunk/vector, similarity search, outbox event.

api/routes.py	FastAPI routes	Public API	Health check và semantic search endpoint.
---------------	----------------	------------	---

3.2 API handler và hàm chính

Bảng 3.2: Handler/hàm quan trọng

File	Hàm/API handler	Input	Output
api.py	list_articles	limit, offset, category, q	Danh sách bài báo
api.py	get_article	article_id, auth optional	Chi tiết bài, categories, reactions
api.py	add_reaction	article_id, reaction body, auth	Tổng reaction mới
api.py	create_comment	article_id, content, auth	Comment đã tạo
ai_podcast/router.py	create_podcast_job	Danh sách bài hoặc rule gợi ý	Job podcast trạng thái 202
article embedding service	process_message	Kafka event bài báo	Kết quả processed, skipped hoặc failed
article embedding service	search_articles	query, limit	Các chunk gần nghĩa nhất
routes.py	search_articles	JSON body semantic search	JSON search response hoặc lỗi 400/503

Các handler validate dữ liệu bằng FastAPI/Pydantic hoặc parser nội bộ. Lỗi nghiệp vụ được chuẩn hóa thành `HTTPException`; lỗi provider hoặc hạ tầng được log và trả mã phù hợp để client/gateway có thể xử lý.

3.3 Bảng cơ sở dữ liệu

Bảng 3.3: Bảng dữ liệu chính

Bảng	Service sở hữu	Mục đích	Cột/constraint quan trọng
news_sources	article	Nguồn RSS	domain, rss_url unique
articles	article	Bài báo gốc	original_url unique, index published_at
categories	article	Taxonomy bài báo	slug unique, is_active
category_articles	article	Quan hệ bài-category	Unique theo arti- cle/category/source, primary category partial index
article_stats	article	Lượt xem tổng hợp	PK article_id
article_interactions	article	Reaction người dùng	Unique article/user, type LIKE/LOVE/DISLIKE
article_comments	article	Bình luận	article_id, user_id, content
article_metrics	article	Metric đọc bài	reading_time _seconds
article_embedding_documents	embedding	Bản mirror để embed bài	article_id unique, content_hash, sync_status
article_embedding_chunks	embedding	Chunk nội dung bài	Unique document_id, chunk_index
article_chunk_embeddings	embedding	Vector từng chunk	VECTOR (1536) , HNSW cosine index
article_document_embeddings	embedding	Vector tổng hợp bài	VECTOR (1536) , HNSW cosine index
category_embedding_documents	embedding	Document category	category_id unique, semantic_text
category_embeddings	embedding	Vector category	Unique document/model/version
article_category_matches	embedding	Kết quả gán category	Unique arti- cle/category/model/version, score, rank
embedding_jobs	embedding	Theo dõi xử lý event	Unique theo tar- get/hash/model/version
outbox_events	embedding	Publish event sync category	status, attempts, available_at

3.4 API nội bộ và API gọi ngoài

Bảng 3.4: API trong phạm vi phụ trách

API	Loại	Method	Endpoint/URL	Auth
Health article	Nội bộ	GET	/healthz	Không
Feed bài báo	Public qua gateway	GET	/api/v1/article	Optional
Category	Public qua gateway	GET	/api/v1/article/categories	Không
Category yêu thích	Protected	GET/PUT	/api/v1/article/me/preferences/categories	Có
Chi tiết bài	Public qua gateway	GET	/api/v1/article/{article_id}	Optional
Reaction	Protected	GET/POST	/api/v1/article/{id}/reactions	Có khi ghi
Comment	Protected/Public	GET/POST	/api/v1/article/{id}/comments	Có khi ghi
Metric đọc bài	Protected	POST	/api/v1/article/{id}/metric	Có
Podcast job	Protected	POST/GET	/api/v1/article/podcast-jobs	Có
Health embedding	Public	GET	/api/v1/public/embedding/healthz	Không
Semantic search	Public	POST	/api/v1/public/embedding/articles/search	Không
Gemini Embedding	Gọi ngoài	POST	Google GenAI Embedding API	API key
Gemini/TTS/Search	Gọi ngoài	POST/GET	Google GenAI, Brave Search, MinIO	API key/credential

CHƯƠNG 4

TRIỂN KHAI VÀ KIỂM THỬ

4.1 Yêu cầu môi trường

Môi trường local cần Docker, Docker Compose, Python 3.12, PostgreSQL/pgvector, Kafka, Debezium, Redis và tài khoản/API key cho Gemini nếu chạy embedding hoặc podcast AI thật. Các service chính dùng cổng: API Gateway 8080, article service 8084, embedding service 8088, Article DB 5433, Embedding DB 5434, Kafka UI 8090.

4.2 Các bước triển khai

Quy trình triển khai local được thực hiện theo thứ tự sau để bảo đảm database, message broker và service nghiệp vụ sẵn sàng trước khi kiểm thử chức năng.

1. **Chuẩn bị mã nguồn và công cụ.** Clone repository, mở thư mục gốc dự án, kiểm tra Docker/Docker Compose và Python 3.12 đã được cài đặt.
2. **Cấu hình biến môi trường.** Tạo hoặc cập nhật file môi trường tương ứng với `article_service` và `embedding_service`. Các biến tối thiểu cần có:

```
ARTICLE_DATABASE_URL=postgresql://postgres:postgres@article-  
postgres:5432/pody_article  
EMBEDDING_DATABASE_URL=postgresql://postgres:postgres@embedding-  
postgres:5432/pody_embedding  
GOOGLE_API_KEY=<your-google-api-key>  
EMBEDDING_GEMINI_API_KEYS=<key1,key2>  
GEMINI_EMBEDDING_MODEL=gemini-embedding-001  
EMBEDDING_OUTPUT_DIMENSIONS=1536  
ARTICLE_CHUNK_TARGET_CHARS=1400  
ARTICLE_CHUNK_OVERLAP_CHARS=180
```

```
ARTICLE_CATEGORY_SYNC_TOPIC=article.category.matches.generated
```

3. **Khởi động hạ tầng nền.** Chạy PostgreSQL cho article, PostgreSQL cho embedding, Redis, Kafka và Kafka UI.
4. **Khởi tạo schema dữ liệu.** Chạy `article-db-init` để tạo bảng bài báo/category/comment/metric và chạy `embedding-db-init` để bật extension `vector`, tạo bảng `document/chunk/vector/job`.
5. **Đăng ký Debezium connector.** Chạy `debezium-connect` và `debezium-register` để theo dõi thay đổi từ Article DB và phát event sang Kafka.
6. **Khởi động service nghiệp vụ.** Chạy `article-service` và `embedding-service`; nếu cần đi qua gateway thì khởi động thêm API Gateway.
7. **Kiểm tra trạng thái sau triển khai.** Gọi các endpoint health check, kiểm tra connector Debezium và mở Kafka UI để xem topic/event.

Lệnh Docker Compose dùng cho toàn bộ luồng trên:

```
docker compose up --build kafka article-postgres article-db-
init redis `
embedding-postgres embedding-db-init debezium-connect
debezium-register `
article-service embedding-service kafka-ui
```

Sau khi container chạy ổn định, kiểm tra health check:

```
curl http://localhost:8084/healthz
curl http://localhost:8088/healthz
curl http://localhost:8088/api/v1/public/embedding/healthz
curl http://localhost:8083/connectors
```

4.3 Các bước kiểm thử

Kiểm thử được chia thành ba mức: unit test cho từng service, kiểm thử API thủ công và kiểm thử tích hợp qua Kafka/Debezium.

Kiểm thử tự động

1. **Cài dependency test.** Vào từng service, cài các thư viện Python cần thiết theo file requirement của service.
2. **Chạy test article service.** Nhóm test này kiểm tra API bài báo, validation, truy vấn dữ liệu, reaction, comment và metric.
3. **Chạy test embedding service.** Nhóm test này kiểm tra xử lý event, chia chunk, gọi provider embedding dạng mock, lưu vector và semantic search.
4. **Đọc kết quả.** Test đạt khi toàn bộ case ở trạng thái passed; nếu có failed, xem traceback để xác định lỗi route, schema, mock provider hoặc kết nối DB.

```
cd server/article_service
python -m pytest tests

cd ../embedding_service
python -m pytest tests
```

Kiểm thử API thủ công

Sau khi test tự động đạt, chạy một kịch bản API ngắn để xác nhận chức năng hoạt động khi service đã được triển khai:

1. Gọi GET `/api/v1/article?limit=10` để kiểm tra danh sách bài báo trả về đúng format.
2. Gọi GET `/api/v1/article/{id}` để kiểm tra chi tiết bài và xác nhận `view_count` tăng.
3. Gửi reaction, comment và metric với header `X-Auth-User-ID`; kiểm tra response và dữ liệu ghi vào DB.
4. Tạo podcast báo qua POST `/api/v1/article/podcast-jobs`; theo dõi trạng thái job từ pending đến hoàn tất hoặc lỗi.

5. Gọi semantic search để kiểm tra truy vấn ngữ nghĩa:

```
curl -X POST http://localhost:8088/api/v1/public/embedding/
  articles/search `
-H "Content-Type: application/json" `
-d "{\"query\":\"tin tức công nghệ AI\", \"limit\":5}"
```

Kiểm thử tích hợp event

1. Thêm hoặc cập nhật một bài báo trong Article DB.
2. Kiểm tra Debezium phát event lên Kafka topic `article.embedding.requested`.
3. Kiểm tra `embedding_service` consume event, tạo chunk, sinh embedding và lưu kết quả vào Embedding DB.
4. Kiểm tra event `article.category.matches.generated` được publish sau khi so khớp category.
5. Kiểm tra `article_service` nhận event và cập nhật bảng liên kết category-bài báo.

4.4 Lưu ý triển khai

- Không commit API key, token, password thật lên GitHub.
- `article-db-init` phải chạy trước Debezium register và trước service chính.
- `embedding-db-init` cần bật `extension vector`; nếu reset DB phải cân nhắc mất dữ liệu vector.
- Embedding service chỉ chạy consumer khi database, Kafka topic và Gemini provider sẵn sàng.
- Nếu thiếu Gemini key, health có thể ở trạng thái degraded; semantic search không hoạt động đầy đủ.

- Gateway cần route đúng tới `article-service:8084`; endpoint `embedding public` có thể gọi trực tiếp qua service hoặc cấu hình thêm route gateway.

CHƯƠNG 5

THÔNG TIN SOURCE CODE

5.1 Thông tin source code

Bảng 5.1: Thông tin repository

Nội dung	Thông tin
Link GitHub/source code	https://github.com/pody-team/pody-app.git
Branch sử dụng để chấm	main
Commit hash tham chiếu	8414c0e
Cách chạy code	Docker Compose theo hướng dẫn ở Chương 4
Tài khoản demo	Dùng tài khoản do nhóm chuẩn bị trong kịch bản demo hoặc header X-Auth-User-ID khi test service riêng

5.2 Các file cá nhân liên quan

Bảng 5.2: File/folder liên quan trực tiếp

STT	File/Folder	Vai trò	Chức năng
1	server/article_service/	Service chính về báo	API bài báo, category, tương tác, crawler
2	server/article_service/ai_podcast/	Module podcast báo	Job AI podcast, script, TTS, storage
3	server/embedding_service/	Service embedding	Kafka consumer, pgvector, semantic search
4	server/sql/services/article_service.sql	Migration/schema	Bảng bài báo, category, tương tác
5	server/sql/services/embedding_service.sql	Migration/schema	Bảng document, chunk, vector, job, outbox
6	docker-compose.yml	Triển khai local	Khai báo container, env, dependency, health check
7	server/article_service/tests/	Test	Route, integration, category projection, podcast config
8	server/embedding_service/tests/	Test	Chunking, runtime, search contract, embedding service

KẾT LUẬN

Phần `article_service` và `embedding_service` hoàn thiện luồng báo của Pody từ thu thập bài viết, hiển thị nội dung, ghi nhận tương tác đến tìm kiếm ngữ nghĩa và gán category tự động. Hai service được triển khai theo kiến trúc tách lớp, có schema riêng, có event pipeline qua Kafka/Debezium và có hướng dẫn chạy/test để kiểm chứng lại chức năng.